

Base de datos de FastKote

Proyecto: FastKote - Sistema de generación de cotizaciones

Cuaderno: Construcción e implementación

Versión analizada: fastkote-fullstack-layered-v2 (2).zip

1. Propósito de la base de datos

La base de datos de FastKote fue construida para almacenar de forma persistente la información necesaria para operar el proceso de cotización del negocio “Chichi Está de Fiesta”. Su diseño se basa en un modelo relacional que permite mantener integridad entre usuarios, empleados, roles, clientes, paquetes, cotizaciones, rubros y reservas de calendario.

Esta estructura responde a los requisitos del SRS V1.0 relacionados con autenticación, cotizaciones, clientes, paquetes, servicios y administración del sistema. En especial, soporta **RF-01 Autenticación y acceso al sistema**, **RF-02 Gestión de cotizaciones**, **RF-03 Gestión de clientes**, **RF-04 Gestión de paquetes y servicios** y **RF-06 Administración del sistema**.

2. Tecnología usada

Elemento	Tecnología
Motor de base de datos	PostgreSQL
ORM	Prisma ORM
Script de creación	database/01_schema.sql
Script de carga inicial	database/02_seed.sql
Modelo ORM	backend/prisma/schema.prisma
Contenedor local	Docker Compose

Configuración local del contenedor:

Base de datos: fastkote_db
Usuario: fastkote_user
Contraseña: fastkote_pass
Puerto: 5432

3. Archivos relacionados con la base de datos

backend/prisma/schema.prisma
database/01_schema.sql
database/02_seed.sql
docker-compose.yml
backend/src/shared/prisma/prisma.ts

Archivo	Función
schema.prisma	Define modelos, enums, relaciones y mapeo ORM.
01_schema.sql	Crea extensión UUID, enums, tablas, claves foráneas e índices.
02_seed.sql	Carga datos iniciales para pruebas y operación académica.
docker-compose.yml	Levanta PostgreSQL en entorno local.
prisma.ts	Expone instancia compartida de Prisma Client.

4. Tipos ENUM definidos

La base de datos usa enums para controlar estados y tipos válidos. Esto reduce inconsistencias y mejora la integridad de datos.

Enum	Valores	Uso
employee_status	ACTIVE , INACTIVE	Estado del empleado.
client_status	ACTIVE , INACTIVE	Estado del cliente.
client_type	NATURAL , JURIDICAL	Tipo de cliente.
quote_status	DRAFT , SENT , ACCEPTED , REJECTED , EXPIRED	Ciclo de vida de la cotización.
reservation_status	BLOCKED , RELEASED	Estado de reserva en calendario.

Relación con reglas de negocio

Regla / CU	Implementación en BD
RN-02 Cierre requiere estado previo Enviada	Enum quote_status ; la validación estricta debe reforzarse en lógica de aplicación.
RN-03 Solo Aceptada bloquea fecha	Tabla event_reservations con estado BLOCKED cuando la cotización es aceptada.
CU-SGC-L1-15 Actualizar Estado Cotización	Campo quotes.status y tabla event_reservations .

5. Tablas implementadas

La base de datos construida contiene las siguientes tablas principales:

employees
roles
employee_roles
users
clients
catalog_packages

catalog_items

quotes

quote_items

event_reservations

6. Tabla employees

Propósito: almacenar la información del personal del negocio.

Trazabilidad: RF-06 Administración del sistema / CU-SGC-Lo-05 Administrar empleados / CU-SGC-L1-01 Registrar Empleado / CU-SGC-L1-02 Editar Empleado / CU-SGC-L1-04 Desactivar Empleado.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador único del empleado.
identification	VARCHAR(20) UNIQUE	Cédula o identificación.
first_name	VARCHAR(80)	Nombres.
last_name	VARCHAR(80)	Apellidos.
email	VARCHAR(120) UNIQUE	Correo.
phone	VARCHAR(20)	Teléfono.
address	VARCHAR(180)	Dirección.
position	VARCHAR(80)	Cargo operativo.
status	employee_status	Estado activo o inactivo.
created_at	TIMESTAMP	Fecha de creación.
updated_at	TIMESTAMP	Fecha de actualización.

Relaciones:

- Un empleado puede tener un usuario asociado en `users` .
 - Un empleado puede tener varios roles mediante `employee_roles` .
 - Un empleado puede crear clientes y cotizaciones.
-

7. Tabla `roles`

Propósito: definir los perfiles de acceso del sistema.

Trazabilidad: RF-06 Administración del sistema / CU-SGC-L1-05 Asignar rol de sistema a empleado / CU-SGC-L1-06 Consultar rol de trabajo.

Campos principales:

Campo	Tipo	Descripción
<code>id</code>	UUID	Identificador del rol.
<code>name</code>	VARCHAR(60) UNIQUE	Nombre del rol, por ejemplo <code>Admin</code> o <code>Empleado</code> .
<code>description</code>	VARCHAR(180)	Descripción del rol.

Relaciones:

- Se relaciona con `employees` mediante la tabla intermedia `employee_roles` .
-

8. Tabla `employee_roles`

Propósito: resolver la relación muchos a muchos entre empleados y roles.

Trazabilidad: RF-06 / CU-SGC-L1-05 Asignar rol de sistema a empleado.

Campos principales:

Campo	Tipo	Descripción
employee_id	UUID FK	Referencia a employees.id .
role_id	UUID FK	Referencia a roles.id .

Clave primaria compuesta:

(employee_id, role_id)

Esto evita duplicar la misma asignación de rol para un empleado.

9. Tabla users

Propósito: almacenar credenciales de acceso y control de login.

Trazabilidad: RF-01 Autenticación y acceso al sistema / CU-SGC-Lo-01 Autenticar Usuario.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador del usuario.
username	VARCHAR(60) UNIQUE	Nombre de usuario para login.
password_hash	VARCHAR(255)	Contraseña cifrada/hasheada.
failed_attempts	INTEGER	Intentos fallidos acumulados.
locked_until	TIMESTAMP NULL	Bloqueo temporal por intentos fallidos.
is_active	BOOLEAN	Indica si el usuario está habilitado.
employee_id	UUID UNIQUE FK	Relación con empleado.
created_at	TIMESTAMP	Fecha de creación.

Campo	Tipo	Descripción
updated_at	TIMESTAMP	Fecha de actualización.

Reglas soportadas:

- El login se vincula a un empleado existente.
- El usuario puede ser bloqueado temporalmente tras intentos fallidos.
- El acceso depende de que el usuario y el empleado estén activos.

10. Tabla clients

Propósito: almacenar los clientes usados para generar cotizaciones.

Trazabilidad: RF-03 Gestión de clientes / CU-SGC-Lo-04 Administrar Cliente / CU-SGC-L1-07 Registrar Cliente / CU-SGC-L1-08 Actualizar datos del cliente / CU-SGC-L1-09 Consultar Cliente.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador del cliente.
type	client_type	Cliente natural o jurídico.
full_name	VARCHAR(140)	Nombre completo o razón social.
identification	VARCHAR(20) UNIQUE	Cédula o RUC.
email	VARCHAR(120)	Correo.
phone	VARCHAR(20)	Teléfono.
address	VARCHAR(180)	Dirección.
status	client_status	Estado activo o inactivo.
privacy_consent	BOOLEAN	Consentimiento de tratamiento de datos.

Campo	Tipo	Descripción
created_by_id	UUID FK	Empleado que registró el cliente.
created_at	TIMESTAMP	Fecha de creación.
updated_at	TIMESTAMP	Fecha de actualización.

Relaciones:

- Un cliente puede tener muchas cotizaciones.
- Un cliente puede estar asociado al empleado que lo registró.

11. Tabla catalog_packages

Propósito: almacenar paquetes comerciales disponibles para cotizar.

Trazabilidad: RF-04 Gestión de paquetes y servicios / CU-SGC-Lo-07 Administrar Paquetes / CU-SGC-L1-20 Registrar Paquete / CU-SGC-L1-22 Consultar Paquete.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador del paquete.
name	VARCHAR(120) UNIQUE	Nombre del paquete.
event_types	TEXT[]	Tipos de evento a los que aplica.
base_price	NUMERIC(10,2)	Precio fijo del paquete.
price_per_child	NUMERIC(10,2)	Precio por niño/persona.
min_children	INTEGER	Mínimo de personas/niños.
capacity_max	INTEGER	Capacidad máxima sugerida.
active	BOOLEAN	Indica si el paquete está disponible.

Relaciones:

- Un paquete puede tener varios ítems en `catalog_items` .
- Se usa para crear cotizaciones base o por persona.

12. Tabla `catalog_items`

Propósito: almacenar ítems asociados a paquetes del catálogo.

Trazabilidad: RF-04 Gestión de paquetes y servicios / CU-SGC-Lo-09 Administrar servicios/productos / CU-SGC-L1-24 Asignar servicios/productos a paquete.

Campos principales:

Campo	Tipo	Descripción
<code>id</code>	UUID	Identificador del ítem.
<code>package_id</code>	UUID FK	Paquete al que pertenece.
<code>name</code>	VARCHAR(120)	Nombre del servicio/producto.
<code>category</code>	VARCHAR(80)	Categoría del ítem.
<code>unit</code>	VARCHAR(40)	Unidad de medida.
<code>base_price</code>	NUMERIC(10,2)	Precio base del ítem.

Relaciones:

- Pertenece opcionalmente a un paquete.
- Si se elimina el paquete, sus ítems asociados se eliminan por `ON DELETE CASCADE` .

Observación:

En la implementación actual, esta tabla funciona como catálogo básico de ítems asociados a paquetes. No representa aún un módulo completo de recetas técnicas ni insumos detallados.

13. Tabla quotes

Propósito: almacenar las cotizaciones generadas.

Trazabilidad: RF-02 Gestión de cotizaciones / CU-SGC-Lo-03 Administrar Cotización / CU-SGC-L1-11 Generar Cotización / CU-SGC-L1-13 Editar Cotización / CU-SGC-L1-15 Actualizar Estado Cotización.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador de la cotización.
code	VARCHAR(30) UNIQUE	Código de cotización, por ejemplo FK-2026-123456 .
client_id	UUID FK	Cliente asociado.
event_date	DATE	Fecha del evento.
event_type	VARCHAR(80)	Tipo de evento.
status	quote_status	Estado: borrador, enviada, aceptada, rechazada o vencida.
version	INTEGER	Número de versión de la cotización.
subtotal	NUMERIC(10,2)	Subtotal calculado.
tax	NUMERIC(10,2)	IVA calculado.
discount	NUMERIC(10,2)	Descuento aplicado.
total	NUMERIC(10,2)	Total final.
valid_until	DATE	Fecha de validez de la cotización.
notes	TEXT	Observaciones.
created_by_id	UUID FK	Empleado que creó la cotización.

Campo	Tipo	Descripción
created_at	TIMESTAMP	Fecha de creación.
updated_at	TIMESTAMP	Fecha de actualización.

Relaciones:

- Pertenece a un cliente.
- Puede tener muchos rubros en `quote_items`.
- Puede tener una reserva de evento en `event_reservations`.
- Puede asociarse al empleado creador.

Observación sobre versionamiento:

La tabla incluye el campo `version` y la implementación incrementa ese valor al editar. Sin embargo, no existe una tabla `quote_versions`; por tanto, el historial completo de versiones queda parcialmente implementado respecto al **CU-SGC-L1-13 Editar Cotización**.

14. Tabla `quote_items`

Propósito: almacenar el detalle de rubros de una cotización.

Trazabilidad: RF-02 / CU-SGC-L1-11 Generar Cotización / CU-SGC-L1-12 Generar Cotización Personalizada.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador del rubro.
quote_id	UUID FK	Cotización a la que pertenece.
description	VARCHAR(150)	Descripción del rubro.
category	VARCHAR(80)	Categoría.

Campo	Tipo	Descripción
quantity	INTEGER	Cantidad.
unit_price	NUMERIC(10,2)	Precio unitario.
subtotal	NUMERIC(10,2)	Subtotal del rubro.

Relaciones:

- Muchos rubros pertenecen a una cotización.
- Si se elimina una cotización, sus rubros se eliminan por `ON DELETE CASCADE`.

Uso en Strategy:

- `FixedPackagePricingStrategy` genera un rubro de paquete base.
- `PerChildPricingStrategy` genera rubro por número de niños/personas.
- `CustomPricingStrategy` genera rubros manuales personalizados.

15. Tabla `event_reservations`

Propósito: registrar el bloqueo o liberación de fechas de eventos.

Trazabilidad: CU-SGC-L1-15 Actualizar Estado Cotización / CU-SGC-L1-17 Consultar Calendario de Eventos / RN-03 Solo Aceptada bloquea fecha.

Campos principales:

Campo	Tipo	Descripción
id	UUID	Identificador de la reserva.
quote_id	UUID UNIQUE FK	Cotización asociada.
event_date	DATE	Fecha del evento.
status	reservation_status	BLOCKED o RELEASED.

Campo	Tipo	Descripción
reason	VARCHAR(180)	Motivo del bloqueo/liberación.
created_at	TIMESTAMP	Fecha de creación.

Regla implementada:

- Cuando la cotización pasa a `ACCEPTED` , `AcceptQuoteStatusStrategy` crea o actualiza una reserva en estado `BLOCKED` .
- Cuando la cotización pasa a `REJECTED` o `EXPIRED` , `ReleaseQuoteStatusStrategy` libera la reserva con estado `RELEASED` .
- La consulta de calendario lista reservas con estado `BLOCKED` .

16. Relaciones principales

employees 1 — 0..1 users
employees N — M roles, mediante employee_roles
employees 1 — N clients, mediante created_by_id
employees 1 — N quotes, mediante created_by_id
clients 1 — N quotes
catalog_packages 1 — N catalog_items
quotes 1 — N quote_items
quotes 1 — 0..1 event_reservations

17. Índices creados

El script SQL crea índices para mejorar consultas frecuentes.

Índice	Tabla	Campos	Uso
idx_clients_search	clients	full_name , identification ,	Búsqueda de clientes.

Índice	Tabla	Campos	Uso
		email	
idx_quotes_filters	quotes	client_id , status , event_date	Filtros de cotizaciones.
idx_reservations_date	event_reservations	event_date , status	Consulta de calendario.

18. Trazabilidad BD - RF/CU

RF/CU	Tablas que lo soportan
RF-01 / CU-SGC-Lo-01 Autenticar Usuario	users , employees , roles , employee_roles .
RF-02 / CU-SGC-Lo-03 Administrar Cotización	quotes , quote_items , clients , employees .
CU-SGC-L1-11 Generar Cotización	quotes , quote_items , catalog_packages , catalog_items .
CU-SGC-L1-12 Generar Cotización Personalizada	quotes , quote_items .
CU-SGC-L1-15 Actualizar Estado Cotización	quotes , event_reservations .
CU-SGC-L1-17 Consultar Calendario de Eventos	event_reservations , quotes , clients .
CU-SGC-L1-18 Generar PDF de Cotización	quotes , quote_items , clients .
CU-SGC-L1-19 Enviar Cotización por WhatsApp	quotes , quote_items , clients .

RF/CU	Tablas que lo soportan
RF-03 / CU-SGC-Lo-04 Administrar Cliente	clients .
RF-04 / CU-SGC-Lo-07 Administrar Paquetes	catalog_packages , catalog_items .
RF-06 / CU-SGC-Lo-05 Administrar Empleados	employees , users , roles , employee_roles .

19. Aspectos pendientes de base de datos

La base de datos implementada permite operar el prototipo principal, pero quedan elementos pendientes si se desea cubrir todo el SRS V1.0:

Pendiente	RF/CU afectado
Tabla de promociones y reglas de aplicación	CU-SGC-Lo-02 / CU-SGC-L1-25 a L1-30.
Tablas completas de insumos	RF-05 / CU-SGC-L1-36 a L1-40.
Tabla de recetas técnicas entre servicios/productos e insumos	CU-SGC-Lo-08 / CU-SGC-L1-32.
Tabla independiente de servicios/productos comerciales	RF-04 / CU-SGC-Lo-09.
Tabla de historial de versiones de cotización	CU-SGC-L1-13.
Tabla de auditoría general	Requisito de trazabilidad indicado en entrevistas y SRS.
Validación de dependencias para bajas lógicas	RF-03, RF-04, RF-05.

20. Conclusión

La base de datos de FastKote está construida sobre PostgreSQL con un modelo relacional normalizado para usuarios, empleados, roles, clientes, paquetes, cotizaciones, rubros y reservas de calendario. Esta estructura cubre el núcleo del proceso de cotización definido en el SRS: autenticación, clientes, empleados, cotizaciones, PDF, WhatsApp y calendario.

Sin embargo, para cubrir por completo la visión del SRS V1.0, se recomienda ampliar el modelo con promociones, insumos, recetas técnicas, auditoría y versionamiento histórico formal de cotizaciones.